



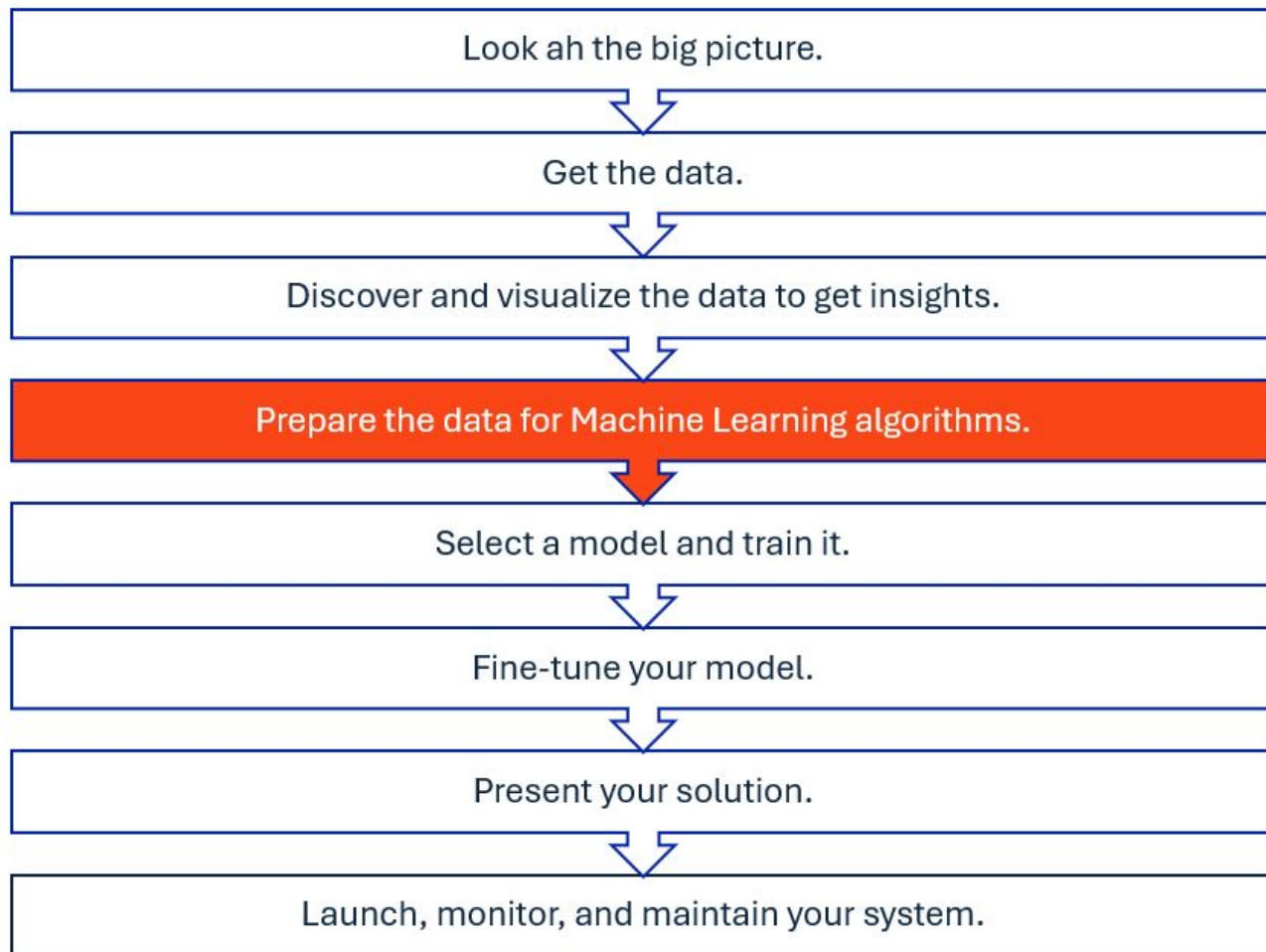
Module 2. Supervised Learning and Experimental Design

Partitioning the data

Data cleaning: Handle missing values and outliers.

Feature scaling and encoding categorical variables

End-to-End Machine Learning Process

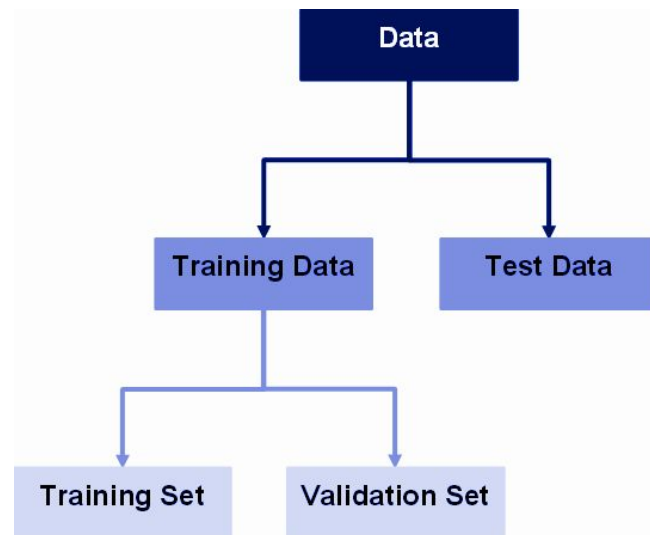




The Test Set

Test Set

- Before carrying experimental design choices such as model selection and fine-tuning the model, it is important that we create a dataset where we can test the model's generalization ability. This is often done by allocating a subset of the samples as a test set.
- The role of the test set is to evaluate the model's performance and generalization. It should not be used at all during the training phase in order to avoid the data snooping bias that often happens.
- We typically partition the data at random into train set and test set. Typically splits are 80/20%.



Test Set

- Scikit-Learn provides a few functions to split datasets into multiple subsets in various ways. The simplest function is `train_test_split()`
- It includes a `random_state` parameter that allows you to set the random generator seed. If you run the program again and don't fix the random state, it will generate a different test set!

```
from sklearn.model_selection import train_test_split

train_test_split?
```

```
train_set, test_set = train_test_split(housing,
                                       test_size=0.2,
                                       random_state=42,
                                       shuffle=True)
```

Test Set

train_set

	longitude	latitude	housing
14196	-117.03	32.71	
8267	-118.16	33.77	
17445	-120.48	34.66	
14265	-117.11	32.69	
2271	-119.80	36.78	
...	...	...	
11284	-117.96	33.78	
11964	-117.43	34.02	
5390	-118.38	34.03	
860	-121.96	37.58	
15795	-122.42	37.77	

16512 rows × 10 columns

test_set

	longitude	latitude	h
20046	-119.01	36.06	
3024	-119.46	35.14	
15663	-122.44	37.80	
20484	-118.72	34.28	
9814	-121.93	36.62	
...	...	...	
15362	-117.22	33.36	
16623	-120.83	35.36	
18086	-122.05	37.31	
2144	-119.76	36.77	
3665	-118.37	34.22	

4128 rows × 10 columns

From the previous correlation results, we saw that median income has a moderate predictive value to predict the median house price:

```
corr_matrix = housing.corr(method='pearson', numeric_only=True)
corr_matrix['median_house_value'].sort_values(ascending=False)
```

```
median_house_value    1.000000
median_income          0.688075
total_rooms            0.134153
housing_median_age     0.105623
households             0.065843
total_bedrooms         0.049686
population             -0.024650
longitude              -0.045967
latitude               -0.144160
Name: median_house_value, dtype: float64
```

Since this feature (or attribute) is so valuable, we may want to maintain the prior probabilities of each "category" of median income in the train and test sets.

Stratification

Process of partitioning the dataset into subsets while preserving the (categorical) classes prior probabilities the same as in the original dataset.

For example, if a class has 20 percent examples in the whole dataset, in all samples drawn from the dataset, it should also have approximately 20 percent examples in each subset.

Test Set

Let's visualize the feature `median_income` to gain more insights:

```
# About 95% of the samples lie in the interval [a,b]
```

```
a = housing['median_income'].mean() - 1.96*housing['median_income'].std(ddof=1)
b = housing['median_income'].mean() + 1.96*housing['median_income'].std(ddof=1)
```

```
a, b
```

```
(np.float64(0.14702043573424994), np.float64(7.594321570079703))
```

```
len(np.where((housing['median_income']>=a)&(housing['median_income']<=b))[0])/len(housing['median_income'])
```

```
0.9570736434108527
```

```
housing['median_income'].describe()
```

```
count    20640.000000
mean      3.870671
std       1.899822
min       0.499900
25%       2.563400
50%       3.534800
75%       4.743250
max       15.000100
Name: median_income, dtype: float64
```

Test Set

Categorize median income in order to performed stratified splitting

```
income_cat = pd.cut(housing['median_income'],
                    bins=[0.,1.5, 3.0, 4.5, 6., np.inf],
                    labels=[1, 2, 3, 4, 5])
```

income_cat

0	5
1	5
2	5
3	4
4	3
..	
20635	2
20636	2
20637	2
20638	2
20639	2

Name: median_income, Length: 20640, dtype: category
Categories (5, int64): [1 < 2 < 3 < 4 < 5]

housing['median_income']

0	8.3252
1	8.3014
2	7.2574
3	5.6431
4	3.8462
...	
20635	1.5603
20636	2.5568
20637	1.7000

Stratified splitting

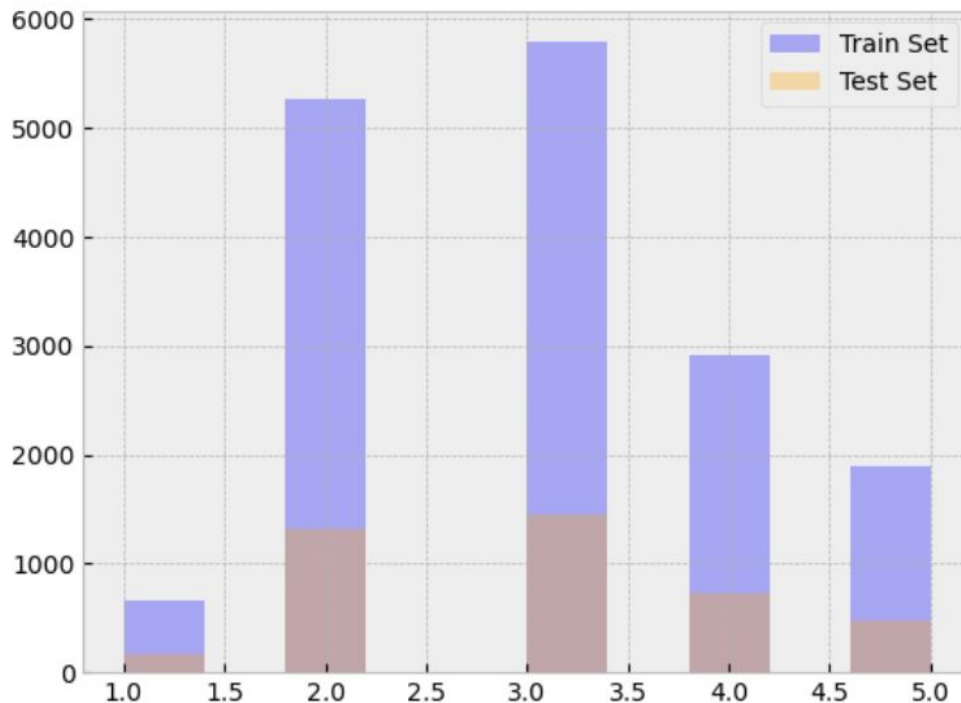
```
train_set, test_set, income_cat_train, income_cat_test = train_test_split(housing, income_cat,
                                  test_size=0.2,
                                  shuffle=True,
                                  random_state=42,
                                  stratify=income_cat)
```

train_set

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_valu
12655	-121.46	38.52	29.0	3873.0	797.0	2237.0	706.0	2.1736	72100.
15502	-117.23	33.09	7.0	5320.0	855.0	2015.0	768.0	6.3373	279600.
2908	-119.04	35.37	44.0	1618.0	310.0	667.0	300.0	2.8750	82700.
14053	-117.13	32.75	24.0	1877.0	519.0	898.0	483.0	2.2264	112500.
20496	-118.70	34.28	27.0	3536.0	646.0	1837.0	580.0	4.4964	238300.
...	...	...	...	...	...	...	...	...	...
15174	-117.07	33.03	14.0	6665.0	1231.0	2026.0	1001.0	5.0900	268500.
12661	-121.42	38.51	15.0	7901.0	1422.0	4769.0	1418.0	2.8139	90400.
19263	-122.72	38.44	48.0	707.0	166.0	458.0	172.0	3.1797	140400.
19140	-122.70	38.31	14.0	3155.0	580.0	1208.0	501.0	4.1964	258100.
19773	-122.14	39.97	27.0	1079.0	222.0	625.0	197.0	3.1319	62700.

Test Set

```
plt.hist(income_cat_train, color='blue', label='Train Set', alpha=0.3)
plt.hist(income_cat_test, color='orange', label='Test Set', alpha=0.3)
plt.legend();
```





The Training Set

Training Set

In preparing your data for ML algorithms it is often convenient and efficient to build functions to automate feature extraction and data cleaning. This provides with:

1. reproducibility on any dataset,
2. reusability,
3. apply the same transformations to the test set,
4. ability to incorporate it with your live system, and
5. carry experiments for design choices.

Training Set

These transformations can be applied to the features only, the target values only or both. For convenience, let's separate them into two variables:

```
# TRAINING: Target Vector and Feature/Input Matrix

t_train = train_set['median_house_value'].copy()

# feature matrix or input matrix
X_train = train_set.drop(labels='median_house_value',axis=1) # axis=1 says it's a column

t_train.shape, X_train.shape

((16512,), (16512, 9))
```

```
# TEST: Target Vector and Feature/Input Matrix

t_test = test_set['median_house_value'].copy()
X_test = test_set.drop(labels='median_house_value', axis=1)

t_test.shape, X_test.shape

((4128,), (4128, 9))
```

Data Cleaning Training Set

In case features have some missing values, you have three options to proceed:

1. Get rid of the corresponding sample.
2. Get rid of the whole feature.
3. Set the values to some value (zero, the mean, the median, etc.).

`pandas` dataframes have readily available methods to accomplish this: `dropna()`, `drop()`, and `fillna()`.

```
X_train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 16512 entries, 12655 to 19773
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   longitude             16512 non-null  float64
1   latitude              16512 non-null  float64
2   housing_median_age    16512 non-null  float64
3   total_rooms           16512 non-null  float64
4   total_bedrooms        16354 non-null  float64
5   population            16512 non-null  float64
6   households            16512 non-null  float64
7   median_income         16512 non-null  float64
8   ocean_proximity       16512 non-null  object
dtypes: float64(8), object(1)
memory usage: 1.3+ MB
```


Data Cleaning Training Set

1. Option 1: drop the rows that are empty

```
# X_train.dropna(subset=['total_bedrooms'])
```

2. Option 2: drop the entire column

```
# X_train.drop('total_bedrooms', axis=1)
```

3. Option 3: fill in missing entries with the statistic median

```
# median = X_train['total_bedrooms'].median()
```

```
# X_train['total_bedrooms'].fillna(median,  
inplace=True)
```

Data Cleaning Training Set

scikit-learn provides a class to handle missing data: `SimpleImputer`.

```
from sklearn.impute import SimpleImputer
```

```
SimpleImputer?
```

```
#sklearn object
```

```
imputer = SimpleImputer(strategy='median')  
imputer
```



SimpleImputer



```
SimpleImputer(strategy='median')
```

Data Cleaning Training Set

Since the median can only be computed on numerical attributes, you need to create a copy of the data without the text feature `ocean_proximity`

```
# Extract numerical features/attributes
```

```
X_train_num = X_train.drop('ocean_proximity', axis=1)
```

```
# Train or computing the median for each attribute/feature  
imputer.fit(X_train_num)
```

```
imputer
```

SimpleImputer

```
SimpleImputer(strategy='median')
```

Data Cleaning Training Set

In `scikit-learn`, all the estimator's hyperparameters are accessible directly via public instance variables (e.g., `imputer.strategy`), and all the estimator's learned parameters are accessible via public instance variables with an underscore suffix (e.g., `imputer.statistics_`).

```
imputer.statistics_
```

```
array([-118.51   ,  34.26   ,  29.   , 2119.   ,  433.   ,
        1164.   ,  408.   ,  3.54155])
```

```
X_train_num.median()
```

```
longitude      -118.51000
latitude        34.26000
housing_median_age  29.00000
total_rooms     2119.00000
total_bedrooms   433.00000
population     1164.00000
households      408.00000
median_income    3.54155
dtype: float64
```

```
# imputing missign values
```

```
X = imputer.transform(X_train_num)
X.shape
```

```
(16512, 8)
```

Handling Text and Categorical Inputs

```
X_train_cat = housing[['ocean_proximity']]
```

```
X_train_cat
```

ocean_proximity	
0	NEAR BAY
1	NEAR BAY
2	NEAR BAY
3	NEAR BAY
4	NEAR BAY
...	...
20635	INLAND
20636	INLAND
20637	INLAND
20638	INLAND
20639	INLAND

20640 rows × 1 columns

Handling Text and Categorical Inputs

Let's convert the categories to numerical representations. There are 2 options:

1. **Integer encoding.** Each category is encoded to a discrete number (1, 2, 3, etc.). The issue with integer encoding is the fact that we will inject an implicit order in the categories ($1 < 2$, $1 < 3$, etc.) that may not reflect the (physical) properties that feature is describing.
2. **One-Hot Encoding.** Each category is encoded as a sparse vector. That vector will have as many entries as there are categories. They will all be zero except at the integer location that category belongs, in which it will have an entry equal to 1. The challenge with one-hot encoding is that these representations can be very sparse and pose challenges later on in the algorithm design and training.
 - **Binary Encoding.** Choose a binary (or other base) encoding to reduce sparsity while not imposing implicit ordering

Handling Text and Categorical Inputs

```
from sklearn.preprocessing import OrdinalEncoder
```

```
OrdinalEncoder?
```

```
ordinal_encoder = OrdinalEncoder()  
ordinal_encoder
```

► OrdinalEncoder ⓘ ?

```
X_train_cat_encoded = ordinal_encoder.fit_transform(X_train_cat)
```

```
# fit_transform should ONLY be used with the TRAINING dataset  
# for the TEST set, we use the TRANSFORM method.
```

```
ordinal_encoder
```

► OrdinalEncoder ⓘ ?

```
X_train_cat_encoded
```

```
array([[3.],  
       [3.],  
       [3.],  
       ...,  
       [1.],  
       [1.],  
       [1.]])
```

```
ordinal_encoder.categories_
```

```
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],  
      dtype=object)]
```

Option 1

Handling Text and Categorical Inputs

Option 2

```
from sklearn.preprocessing import OneHotEncoder
```

OneHotEncoder?

```
cat_encoder = OneHotEncoder()
```

```
cat_encoder
```

OneHotEncoder ⓘ ⓘ

```
OneHotEncoder()
```

```
X_train_cat_1hot = cat_encoder.fit_transform(X_train_cat)
```

```
X_train_cat_1hot
```

```
<20640x5 sparse matrix of type '<class 'numpy.float64'>'
  with 20640 stored elements in Compressed Sparse Row format>
```

```
X_train_cat_1hot.toarray()
```

```
array([[0., 0., 0., 1., 0.],
       [0., 0., 0., 1., 0.],
       [0., 0., 0., 1., 0.],
       ...,
       [0., 1., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       [0., 1., 0., 0., 0.]])
```


Feature Scaling

One of the most important transformations you need to apply to your data is feature scaling. With few exceptions, Machine Learning algorithms don't perform well when the input numerical attributes have very different scales.

There are two common ways to get all attributes to have the same scale:

1. **Min-Max Scaling.** Values are shifted and rescaled so that they end up ranging from 0 to 1.
 - Highly affected by outliers.
2. **Standardization.** First it subtracts the mean value (so standardized values always have a zero mean), and then it divides by the standard deviation so that the resulting distribution has unit variance.
 - Less affected by outliers
 - Gaussianity assumptions

Feature Scaling - Important

- As with all the transformations, it is important to fit the scalers to the training data only, not to the full dataset (including the test set). Only then can you use them to transform the training set and the test set (and new data).
- Note that scaling the target values is generally not required.

Coming Next!

Transformation Pipelines!